

Reinforcement Learning per l'incident response in una rete aziendale simulata

Stefano Deri – Matricola 0001233279
Corso di Intelligenza Artificiale
Laurea Magistrale in Informatica
A.A. 2025/2026

Sommario

Il progetto sviluppa un ambiente di reinforcement learning che simula una piccola rete aziendale sotto attacco: un attaccante si propaga in modo stocastico tra i nodi e un agente difensore deve reagire turno per turno, senza osservare direttamente dove si trovi l'infezione. L'obiettivo è bilanciare sicurezza e continuità operativa, tenendo conto anche del costo delle contromisure. Nell'ambiente vengono confrontate cinque strategie di riferimento e due algoritmi di RL, DQN e PPO, utilizzando un protocollo di valutazione comune e un test di significatività basato su bootstrap appaiato. PPO supera in modo statisticamente significativo la migliore strategia procedurale, mentre DQN, a parità di budget, non batte in modo significativo nemmeno la strategia completamente passiva. L'analisi del comportamento appreso mostra che il risultato di PPO è riconducibile soprattutto a un comportamento ricorrente di contenimento alla sorgente, efficace nel caso tipico ma insufficiente negli episodi difficili osservati; la robustezza rispetto a topologie o punti d'ingresso differenti non è stata verificata sperimentalmente.

Indice

1	Introduzione	4
1.1	Il problema e perché usare il reinforcement learning	4
1.2	Obiettivi e domande di ricerca	4
1.3	Perimetro del progetto	4
1.4	Struttura della relazione	4
2	L'ambiente di simulazione	5
2.1	La rete e i suoi nodi	5
2.2	Lo stato di un nodo e l'osservabilità parziale	5
2.3	Azioni disponibili	6
2.4	L'attaccante	6
2.5	La funzione di reward	6
2.6	Formalizzazione del problema	7
3	Le strategie di riferimento	7
4	Gli agenti: DQN e PPO	7
4.1	Iperparametri e budget	8
4.2	Riproducibilità e gestione dei seed	8
4.3	Andamento degli addestramenti	9
5	Protocollo di valutazione e metriche	10
6	Risultati	10
6.1	Confronto complessivo	10
6.2	Significatività delle differenze	11
6.3	Interpretazione del divario tra DQN e PPO	11
7	Analisi del comportamento appreso	12
7.1	Livello 1 — Comportamento aggregato	12
7.2	Livello 2 — Comportamento rispetto allo stato reale	12
7.3	Livello 3 — Analisi di episodi interi	13
7.4	Sintesi dell'analisi	13
8	Discussione e limiti	13
9	Conclusioni	14
A	Struttura del codice e riproducibilità	15
B	Iperparametri completi	16

1 Introduzione

1.1 Il problema e perché usare il reinforcement learning

Quando una macchina di una rete aziendale viene compromessa, il difensore non conosce con certezza lo stato reale del sistema. Osserva alcuni segnali sospetti, sa che il problema può propagarsi alle macchine vicine e deve decidere rapidamente come intervenire: indagare per capire meglio, isolare una macchina dalla rete, ripulirla e rimetterla in funzione, oppure non intervenire, perché una reazione può causare più danni dell'attacco stesso. Si tratta di una sequenza di decisioni prese sotto incertezza, in cui ogni mossa modifica lo stato del sistema e nessuna informazione è mai completa.

Il problema combina decisioni sequenziali, informazione parziale e conseguenze che si accumulano nel tempo; per questo viene affrontato mediante reinforcement learning. All'agente vengono forniti un ambiente e un meccanismo di ricompensa che valuta le decisioni prese, mentre la strategia di difesa viene appresa dall'esperienza. La domanda di fondo del progetto è se un agente addestrato in questo modo riesca a fare meglio di regole scritte a mano e se la strategia appresa costituisca una difesa effettiva o funzioni soltanto nelle condizioni particolari di questo scenario.

La difficoltà del problema sta nel compromesso tra due obiettivi in contrasto. Un difensore troppo passivo lascia propagare l'infezione; uno troppo aggressivo blocca la rete isolando e riavviando macchine che potrebbero essere sane. Contenere la minaccia mantenendo il disservizio il più basso possibile è più importante che eliminare ogni rischio.

1.2 Obiettivi e domande di ricerca

Il lavoro è organizzato attorno a tre domande, a cui la relazione risponde nelle conclusioni:

1. Un agente RL riesce a superare le strategie difensive di riferimento, in particolare l'euristica?
2. Fra due algoritmi molto diversi, DQN (basato sul valore) e PPO (basato sulla policy), quale si adatta meglio a questo problema, e perché?
3. Che cosa ha appreso effettivamente l'agente migliore: una strategia di difesa robusta o un comportamento efficace soltanto nello scenario considerato?

1.3 Perimetro del progetto

L'ambiente è stato scelto deliberatamente piccolo e controllato: otto nodi, un solo attaccante con propagazione probabilistica, un solo difensore, osservazione parziale, azioni discrete. Restano fuori, per scelta, gli aspetti più complessi del problema reale, come lo scenario multi-agente e le topologie variabili. Un ambiente piccolo e leggibile è stato preferito perché permette di capire da cosa dipende un comportamento (reward, topologia o algoritmo), cosa che un ambiente più ricco renderebbe difficile da interpretare e da verificare.

1.4 Struttura della relazione

L'ambiente è descritto nella Sezione 2. Le Sezioni 3 e 4 presentano rispettivamente le strategie di riferimento e i due agenti RL. La Sezione 5 definisce il protocollo di valutazione e le metriche, la Sezione 6 riporta i risultati e la Sezione 7 analizza il modo in cui gli agenti prendono le decisioni. Chiudono la relazione la discussione dei limiti e le conclusioni. In appendice si trovano la mappa dei file e i comandi per riprodurre i principali risultati riportati.

2 L'ambiente di simulazione

L'ambiente è implementato come ambiente Gymnasium personalizzato, compatibile con Stable-Baselines3. Fin dall'inizio sono stati tenuti separati tre livelli logici facili da confondere: la *configurazione fissa* della rete (ciò che non cambia mai), lo *stato dinamico* dell'episodio (ciò che cambia a ogni turno) e l'*osservazione* fornita all'agente (ciò che l'agente può vedere di quello stato). Tenere separati questi tre livelli ha permesso di mantenere il codice ordinato e verificabile.

2.1 La rete e i suoi nodi

La rete è composta da otto nodi di tre tipi: cinque nodi *utente* (postazioni di lavoro poste alla periferia della rete), due nodi di *servizio* (che collegano le diverse zone) e un nodo *critico* (il più importante, raggiungibile solo attraverso i nodi di servizio). A ciascun tipo è associato un valore che ne quantifica l'importanza: 1 per i nodi utente, 3 per i nodi di servizio, 6 per il nodo critico. Il danno non si misura in base al numero di nodi colpiti, ma alla somma dei loro valori: perdere il nodo critico pesa molto più che perdere un nodo utente.

La topologia è fissa ed è stata scelta in modo da porre al difensore dilemmi reali: i nodi utente si trovano alla periferia, i nodi di servizio costituiscono il passaggio obbligato verso il nodo critico e un attacco che parte da una postazione può, passo dopo passo, raggiungere il nodo critico. La Figura 1 mostra la struttura.

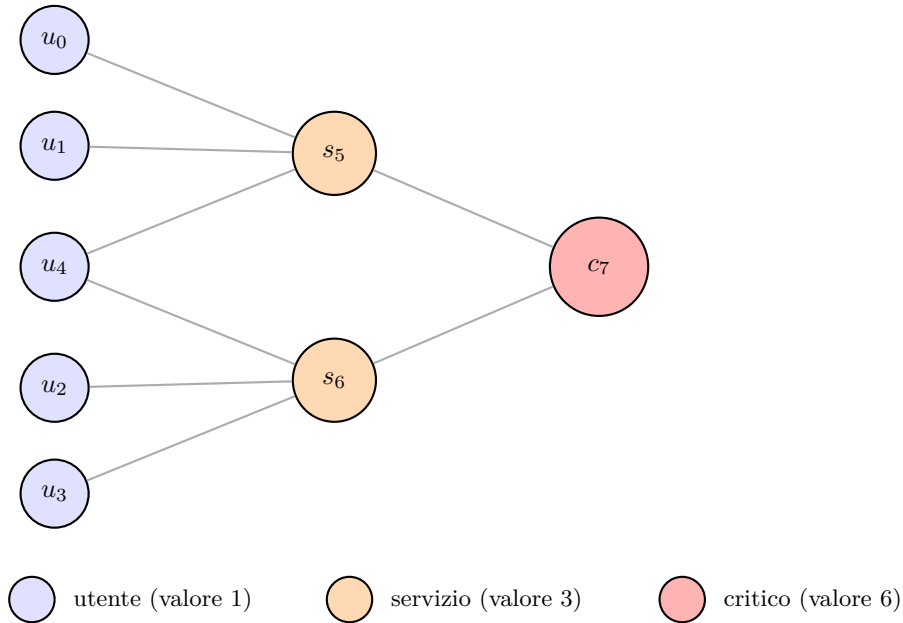


Figura 1: Struttura della rete simulata: gli utenti (periferia) raggiungono il nodo critico solo attraverso i due nodi di servizio.

2.2 Lo stato di un nodo e l'osservabilità parziale

Lo stato reale di ogni nodo è rappresentato da un insieme di cinque informazioni distinte: se è *compromesso*, se è *isolato* dalla rete, se è in fase di *ripristino* (e per quanti turni ancora), se ha un *alert* attivo e l'esito dell'ultima analisi svolta sul nodo, se ancora valido. Questi campi sono tenuti separati perché non si escludono a vicenda: un nodo può essere sano ma generare un falso allarme, oppure essere compromesso e già isolato.

L'osservabilità parziale è l'elemento centrale del problema. Lo stato di compromissione di un nodo non è direttamente incluso nell'osservazione e diventa noto soltanto dopo un'analisi. L'agente osserva invece gli alert, che sono rumorosi: un nodo infetto ne genera uno con probabilità 0,70 e un nodo sano con probabilità 0,10. L'analisi fornisce un'informazione affidabile al costo di un turno; tale informazione decade quando lo stato del nodo cambia. Anche lo stato del ripristino è visibile solo in parte: l'agente sa che un nodo è in fase di ripristino, ma non quanti turni mancano al suo ritorno online. L'osservazione è quindi un vettore di 40 numeri ($8 \text{ nodi} \times 5 \text{ feature}$), tutti normalizzati tra 0 e 1, in cui l'informazione sullo stato di infezione è deliberatamente assente.

2.3 Azioni disponibili

A ogni turno l'agente sceglie una fra 33 azioni. Vi sono quattro tipi di azione mirata, ciascuna applicabile a uno degli otto nodi ($4 \times 8 = 32$), più il non fare nulla:

- **Analyse**: indaga un nodo e ne rivela lo stato reale, al costo di un turno.
- **Isolate**: stacca un nodo dalla rete, interrompendo la propagazione da e verso di esso; il nodo, però, diventa indisponibile.
- **Restore**: avvia il ripristino di un nodo e, se questo è compromesso, lo ripulisce; il nodo resta fuori servizio per due turni durante il ripristino.
- **Reconnect**: ricollega alla rete un nodo isolato.
- **DoNothing**: nessun intervento nel turno corrente.

Un'azione può risultare *non valida* nello stato corrente (per esempio isolare un nodo già isolato). In tal caso non produce alcun effetto sull'ambiente, ma comporta una piccola penalità. Le azioni non valide non devono fornire informazioni sullo stato reale. Per esempio, considerare non valido Restore su un nodo sano permetterebbe all'agente di dedurre lo stato dalla sola penalità, aggirando l'osservabilità parziale. Per questo Restore rimane applicabile anche a un nodo sano, pur comportando un costo.

2.4 L'attaccante

L'attaccante è volutamente semplice: a ogni turno, ogni nodo compromesso può contagiare un vicino non ancora isolato con una certa probabilità. Non segue una strategia e non punta deliberatamente al nodo critico: si propaga ai nodi adiacenti senza una direzione precisa. La semplicità dell'attaccante non riduce l'interesse del problema: anche una propagazione senza direzione mette il difensore di fronte a decisioni non ovvie sotto incertezza.

2.5 La funzione di reward

La reward traduce in un numero il danno complessivo. A ogni turno è negativa e somma tre tipi di costo, pesati per il valore dei nodi coinvolti, più una piccola penalità quando è stata eseguita un'azione non valida:

$$R_t = -\left(\alpha \sum_{i \in \mathcal{C}_t} v_i + \beta \sum_{i \in \mathcal{I}_t} v_i + \gamma \sum_{i \in \mathcal{R}_t} v_i + \eta \cdot \mathbf{1}_{\text{azione non valida}} \right),$$

dove $\mathcal{C}_t$ sono i nodi compromessi, $\mathcal{I}_t$ i nodi isolati (ma non in ripristino), $\mathcal{R}_t$ i nodi in ripristino, v_i il valore del nodo, e i pesi valgono $\alpha = 1,0$, $\beta = 0,5$, $\gamma = 1,0$, $\eta = 0,25$.

I costi riflettono la gravità dei diversi stati. Un nodo compromesso pesa per l'intero suo valore; un nodo isolato pesa la metà, perché comporta comunque un disservizio, ma un disservizio deciso dal difensore e non subito; un nodo in ripristino pesa quanto uno compromesso, perché resta comunque fuori servizio; un'azione non valida comporta solo un piccolo costo fisso.

Questi rapporti tra i coefficienti (compromissione e ripristino a costo pieno, isolamento a metà, azione non valida marginale) definiscono l'importanza relativa dei vari costi e influiscono sul comportamento appreso dall'agente.

Per dare un'idea della scala, una rete interamente compromessa costa -17 per turno, pari alla somma dei valori di tutti i nodi.

2.6 Formalizzazione del problema

Nel suo complesso, l'ambiente è un processo decisionale di Markov sotto osservabilità parziale (formalmente, un POMDP): lo stato reale, ovvero quali nodi siano davvero compromessi, è nascosto, l'osservazione è rumorosa e incompleta, le azioni sono discrete e le transizioni sono stocastiche. Un episodio si conclude con una vittoria se la rete resta pulita per tre turni consecutivi, con una sconfitta totale se tutti i nodi vengono compromessi, oppure si interrompe dopo al massimo 40 turni.

3 Le strategie di riferimento

Prima dell'addestramento sono state definite cinque strategie predefinite da utilizzare come termini di confronto. La reward acquista significato soprattutto se confrontata tra strategie valutate sullo stesso problema. Le politiche considerate rappresentano comportamenti molto diversi, dalla completa passività a una strategia con accesso allo stato reale.

- **DoNothing** — non interviene mai. Costituisce una baseline passiva di riferimento: un agente può essere considerato utile solo se riesce almeno a superarla.
- **PanicIsolation** — isola qualsiasi nodo non appena genera un alert. Rappresenta l'opposto del DoNothing: una difesa che reagisce a ogni segnale isolando tutto, fino a paralizzare la rete.
- **Random** — sceglie le azioni a caso. Serve a misurare quanto conti agire secondo una logica, qualunque essa sia.
- **HeuristicDefender** — una strategia procedurale ragionevole, scritta a mano secondo un ordine di priorità (proteggere il critico, gestire i compromessi noti, analizzare gli alert dubbi, e così via). È il vero termine di confronto, perché rappresenta una difesa procedurale plausibile, del tipo che si potrebbe realizzare senza ricorrere all'apprendimento.
- **Oracle** — legge direttamente lo stato reale dei nodi, violando deliberatamente l'osservabilità parziale, e ripristina solo ciò che è effettivamente compromesso. Non corrisponde a una difesa applicabile nella realtà, perché presuppone informazione perfetta; viene utilizzato come riferimento ideale per valutare il divario rispetto a una strategia che non risente dell'incertezza osservativa.

L'euristica e l'oracle sono i due riferimenti usati più spesso nel seguito: la prima come termine di confronto realistico, il secondo come riferimento ideale con informazione perfetta.

4 Gli agenti: DQN e PPO

Sono stati confrontati due algoritmi che adottano approcci differenti all'apprendimento. DQN stima il valore di ogni azione in ogni stato e sceglie quella con valore più alto; è off-policy

e riutilizza l'esperienza passata attraverso una memoria. PPO apprende direttamente una politica (una distribuzione sulle azioni) e la aggiorna con piccoli passi controllati; è on-policy e normalizza internamente i segnali di vantaggio. I due algoritmi sono stati scelti per confrontare approcci di apprendimento differenti nello stesso ambiente. Entrambi utilizzano una rete neurale completamente connessa (MLP) e sono forniti da Stable-Baselines3. Un adattatore (wrapper) li rende compatibili con la stessa interfaccia delle strategie di riferimento, così da poterli valutare con lo stesso codice.

4.1 Iperparametri e budget

Sono stati usati gli iperparametri di default di Stable-Baselines3, con un'unica eccezione per DQN. Non è stato effettuato un tuning dedicato: l'obiettivo è confrontare le configurazioni standard dei due algoritmi entro lo stesso budget computazionale, senza introdurre una fase separata di ottimizzazione degli iperparametri. Le Tabelle 1 e 2 riassumono le impostazioni principali; l'elenco completo è in Appendice B.

Tabella 1: Iperparametri principali di DQN.

Parametro	Valore
learning rate	1×10^{-4}
dimensione replay buffer	1 000 000
learning starts	5 000 (unica modifica al default)
batch size	32
γ (sconto)	0,99
esplorazione ε	da 1,0 a 0,05 (frazione 0,1)
aggiornamento target	ogni 10 000 passi

Tabella 2: Iperparametri principali di PPO.

Parametro	Valore
learning rate	3×10^{-4}
n_steps (rollout)	2048
batch size	64
n_epochs	10
γ (sconto)	0,99
GAE λ	0,95
clip range	0,2
coeff. entropia	0,0

L'addestramento di entrambi ha usato un budget di 500 000 passi per seed, ripetuto cinque volte con seed diversi (da 0 a 4) per ciascun algoritmo, così da osservare la variabilità tra esecuzioni e ridurre la dipendenza da un singolo seed. In totale sono stati quindi eseguiti dieci addestramenti, tutti salvati con i relativi metadati.

4.2 Riproducibilità e gestione dei seed

Per ogni run viene definito un seed di riferimento `env_seed`, dal quale viene derivato il seed dell'algoritmo secondo la relazione `algo_seed = env_seed + 1000`. Nell'integrazione con Stable-Baselines3, il seed dell'algoritmo viene propagato anche all'ambiente di training al primo reset; i due seed non costituiscono quindi due fonti completamente indipendenti di variabilità durante l'addestramento. L'ambiente utilizzato per la valutazione periodica usa invece un seed distinto, pari a `env_seed + 10000`, così da valutare il modello su scenari differenti da quelli

incontrati durante il training. I parametri della run, il seed dell'algoritmo e le versioni delle librerie sono conservati nei metadati, mentre il seed associato alla run è riportato anche nel nome della relativa directory.

Per DQN la reward è stata divisa per 17, pari alla somma dei valori di tutti i nodi, così da riportarne l'ordine di grandezza intorno all'unità. Una riscalatatura per una costante positiva non modifica l'ordinamento delle policy rispetto al ritorno atteso e riduce la scala numerica del segnale utilizzato durante l'addestramento. Per PPO la reward è stata lasciata nella scala originale; l'algoritmo normalizza internamente i vantaggi utilizzati per aggiornare la policy.

4.3 Andamento degli addestramenti

La differenza tra i due algoritmi è visibile già nelle curve di apprendimento (Figura 2). PPO cresce in modo regolare e i cinque seed convergono su valori vicini, con un picco compreso tra -28 e -20 . DQN è molto più irregolare: oscilla, i seed si disperdono e in diversi casi la prestazione finale è nettamente peggiore del picco raggiunto durante l'addestramento (Tabella 3). La maggiore instabilità di DQN persiste nonostante la normalizzazione della reward e non può quindi essere attribuita alla sola scala del segnale. Le caratteristiche dell'apprendimento off-policy con bootstrap e l'ampiezza dello spazio d'azione costituiscono possibili concause del comportamento osservato, ma il protocollo adottato non consente di isolare il contributo dei singoli fattori.

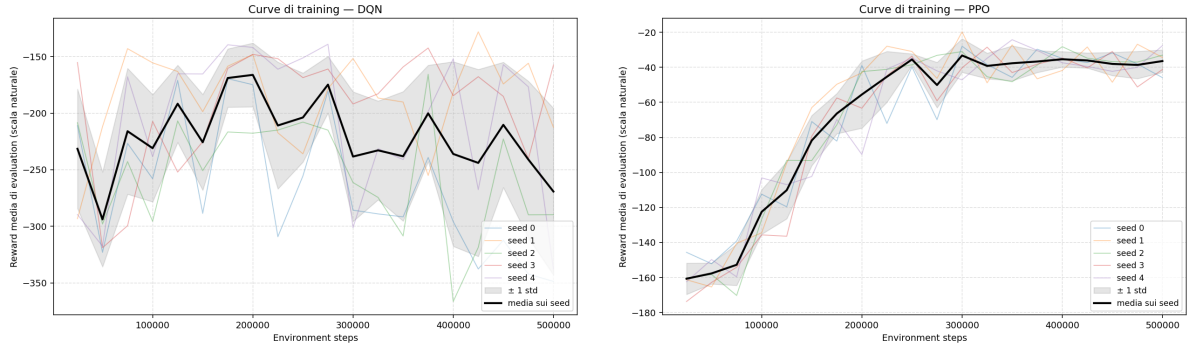


Figura 2: Andamento dell'addestramento: DQN (a sinistra) mostra una maggiore variabilità, mentre PPO (a destra) presenta un andamento più regolare. Si noti che le due scale verticali sono diverse.

Tabella 3: Picco e valore finale della reward durante la valutazione, per seed. I valori sono riportati alla scala naturale; per DQN il divario tra picco e valore finale evidenzia la maggiore instabilità osservata durante l'addestramento.

DQN			PPO		
seed	picco	finale	seed	picco	finale
0	-169,6	-348,6	0	-27,9	-45,8
1	-128,2	-212,5	1	-19,7	-34,3
2	-165,7	-289,7	2	-28,2	-33,0
3	-142,5	-157,6	3	-28,5	-41,2
4	-139,2	-337,7	4	-24,2	-27,6

Per ciascun algoritmo è stato scelto un seed di riferimento (DQN seed 1, PPO seed 3) come migliore fra i cinque sugli stessi 50 episodi del confronto finale, secondo la reward media. La selezione del migliore tra più seed può favorire in parte il risultato riportato per gli agenti RL; per verificare che il confronto non dipenda esclusivamente da questa scelta sono state

considerate anche le prestazioni degli altri seed. Per PPO, le reward medie sui medesimi episodi variano da $-39,52$ a $-26,82$, e anche il seed con il risultato peggiore mantiene un ampio margine rispetto all'euristica ($-104,28$). Inoltre, la valutazione periodica di Stable-Baselines3, effettuata su scenari generati con un seed distinto da quello del confronto finale, seleziona per DQN lo stesso seed e per PPO il seed 1, che sul confronto finale ottiene comunque una reward media di $-37,61$. Il vantaggio osservato per PPO non dipende quindi esclusivamente dalla scelta del seed 3. La selezione del seed di riferimento sugli stessi episodi introduce comunque un bias di selezione. Inoltre, gli intervalli bootstrap quantificano la variabilità tra episodi condizionatamente al modello selezionato e non incorporano la variabilità dovuta alla scelta del seed di addestramento.

5 Protocollo di valutazione e metriche

Tutte le strategie, sia quelle di riferimento sia quelle addestrate, sono valutate con la stessa procedura: 50 episodi generati da un seed condiviso (42). L'uso di un seed condiviso consente di accoppiare gli episodi corrispondenti tra le diverse strategie e riduce la variabilità dovuta alla diversa difficoltà degli scenari. Il confronto è quindi di tipo *appaiato*: le differenze vengono calcolate tra risultati ottenuti su episodi corrispondenti, riducendo l'effetto della variabilità tra scenari.

La reward media da sola può nascondere differenze importanti (due strategie con punteggio simile possono avere esiti molto diversi); vengono quindi considerate anche altre metriche complementari:

- **reward media** (e deviazione standard): la misura principale;
- **tasso di vittorie** e di **sconfitte totali**: perché la fine di un episodio può significare tanto una vittoria (infezione contenuta) quanto una sconfitta (rete interamente compromessa), ed è essenziale distinguere i due casi;
- **tasso di azioni non valide**: quanto spesso l'agente esegue mosse non applicabili nello stato in cui si trova;
- **episodi con nodo critico compromesso**: quante volte l'attacco raggiunge il nodo critico;
- **valore compromesso medio**: il valore operativo complessivo dei nodi compromessi, mediato sui passi dell'episodio e poi sugli episodi.

Infine, per non confondere una differenza reale con una fluttuazione casuale, la significatività statistica è misurata con un *bootstrap appaiato*: i 50 scenari vengono ricampionati molte volte e si costruisce un intervallo di confidenza al 95% sulla differenza di reward tra due strategie. Se l'intervallo non contiene lo zero, la differenza è significativa; in caso contrario non lo è.

6 Risultati

6.1 Confronto complessivo

La Tabella 4 riassume i risultati del confronto; ogni colonna riporta una metrica diversa.

PPO ottiene il secondo miglior risultato dopo l'oracle, che dispone però di informazione perfetta. Con una reward media di $-26,82$ e l'88% di vittorie, PPO supera l'euristica di riferimento. DQN si colloca invece sotto l'euristica, con zero vittorie e un tasso di azioni non valide del 49%: quasi una mossa su due non produce alcun effetto. Nei 50 episodi considerati, DoNothing termina sempre con l'intera rete compromessa, mentre PanicIsolation mantiene la rete fortemente indisponibile fino alla conclusione dell'episodio e ottiene la reward media peggiore.

Tabella 4: Confronto finale su 50 episodi (seed 42). Legenda: win/wipe = vittorie / sconfitte totali; inv = azioni non valide; crit = episodi con critico compromesso; comp_val = valore operativo medio compromesso per passo. Reward in scala naturale.

strategia	reward	dev. std	passi	win%	wipe%	inv%	crit%	comp_val
oracle (info perfetta)	-2,00	0,00	5,0	100	0	0	0	0,00
PPO	-26,82	55,58	11,4	88	12	12,4	12	1,30
heuristic	-104,28	155,83	14,8	72	22	0	32	3,35
DQN	-141,22	142,75	33,3	0	36	49,0	42	4,48
do_nothing	-160,64	82,22	16,0	0	100	0	100	9,99
random	-352,12	206,97	29,9	16	32	28,4	74	8,03
panic_isolation	-364,11	122,24	40,0	0	0	0	4	2,45

6.2 Significatività delle differenze

Una differenza tra le medie non basta, da sola, a dimostrare che una strategia è migliore: è stato quindi verificato quali differenze siano supportate dal bootstrap appaiato (Tabella 5).

Tabella 5: Confronti appaiati con intervallo di confidenza al 95% sulla differenza di reward. Δ = media(A) - media(B). “sig.” = lo zero è fuori dall’intervallo (differenza significativa al 5%); “n.s.” = non significativa.

A	B	Δ	CI 95%	
PPO	heuristic	+77,47	[+36,85, +120,49]	sig.
PPO	DQN	+114,41	[+72,19, +159,36]	sig.
PPO	do_nothing	+133,82	[+107,29, +161,42]	sig.
DQN	heuristic	-36,94	[-90,94, +16,62]	n.s.
DQN	do_nothing	+19,42	[-27,58, +65,10]	n.s.

Il bootstrap appaiato fornisce una risposta quantitativa alle prime due domande di ricerca. PPO mostra un vantaggio statisticamente significativo rispetto all’euristica ($\Delta = +77,47$, con l’intervallo interamente sopra lo zero), oltre che rispetto a DQN e alla strategia passiva. Per DQN, invece, non emerge un vantaggio statisticamente significativo né rispetto all’euristica né rispetto a DoNothing: nel secondo confronto la differenza di +19,42 presenta un intervallo di confidenza che comprende lo zero. Nel protocollo considerato, PPO mostra quindi un miglioramento statisticamente supportato rispetto alle strategie di riferimento, mentre per DQN non emerge un miglioramento significativo rispetto alla strategia passiva.

6.3 Interpretazione del divario tra DQN e PPO

Le curve di apprendimento e i risultati finali mostrano una differenza marcata tra i due algoritmi: DQN presenta una maggiore variabilità tra seed, mentre PPO raggiunge prestazioni più omogenee. Le differenze nei rispettivi meccanismi di apprendimento, insieme all’ampiezza dello spazio d’azione, offrono possibili chiavi di lettura del divario osservato; gli esperimenti svolti non consentono tuttavia di isolare una singola causa.

DQN è stato inoltre valutato con iperparametri sostanzialmente di default e senza un tuning dedicato. I risultati non permettono quindi di concludere che l’algoritmo sia inadatto in generale. Nel protocollo adottato, PPO risulta più efficace e più stabile.

7 Analisi del comportamento appreso

Le sezioni precedenti hanno confrontato le prestazioni aggregate. L'analisi seguente esamina invece le decisioni degli agenti, con l'obiettivo di verificare se un punteggio elevato corrisponda a un comportamento robusto o a una regola efficace soprattutto nello scenario considerato. L'analisi si basa su un fatto: durante la valutazione è possibile leggere dall'ambiente lo stato reale dei nodi, che l'agente invece non conosceva. Si può quindi confrontare ogni sua mossa con la situazione effettiva — per esempio se abbia ripristinato un nodo in realtà sano, o ignorato un nodo critico in realtà infetto. Tutte le misure che seguono sono calcolate sugli stessi 50 episodi del confronto finale, quindi si basano esattamente sui numeri già presentati.

7.1 Livello 1 — Comportamento aggregato

Il primo livello di analisi riguarda la distribuzione delle azioni (Tabella 6). Emergono due profili opposti. PPO utilizza soprattutto Analyse e Restore e presenta una quota contenuta di azioni non valide. DQN presenta invece un tasso di azioni non valide vicino al 50%, dovuto in larga parte a tentativi di riconnettere nodi che non erano isolati. Questo elevato tasso indica che la policy seleziona frequentemente azioni non applicabili allo stato corrente.

Tabella 6: Distribuzione delle azioni sui 50 episodi. PPO concentra le proprie azioni soprattutto su Analyse e Restore; DQN presenta una quota elevata di azioni non valide. Il tasso di azioni non valide è qui calcolato in forma aggregata (azioni non valide sul totale delle azioni); l'inv% della Tabella 4 è invece la media per episodio, da cui la lieve differenza per PPO (16% contro 12,4%).

	PPO	DQN
mosse totali	569	1667
Analyse	313	74
Restore	165	773
azioni non valide	91	820
Isolate / Reconnect / DoNothing validi	0	0
tasso di azioni non valide (aggregato)	16%	49%

Tra le azioni valide, nessuno dei due agenti utilizza Isolate, Reconnect o DoNothing. Il comportamento effettivo si concentra quindi su Analyse e Restore. Resta da capire con quale criterio queste due azioni vengano scelte, oggetto del livello di analisi successivo.

7.2 Livello 2 — Comportamento rispetto allo stato reale

A questo livello emerge il risultato più importante dell'analisi. Sono stati distinti i Restore preceduti da un'analisi che confermava il compromesso da quelli eseguiti senza una precedente conferma. Per PPO il risultato è netto: il **100% dei Restore** è eseguito senza una precedente conferma tramite Analyse. PPO utilizza Analyse in più della metà delle proprie mosse; queste analisi si concentrano sui nodi di servizio, mentre i Restore vengono eseguiti in risposta agli alert.

Il comportamento osservato segue una regola ricorrente: quando una postazione utente genera un alert, PPO tende a ripristinarla immediatamente. Nel presente ambiente questa scelta può contenere l'infezione all'origine, poiché il primo nodo infetto è sempre una postazione utente. La policy non usa però le analisi per confermare gli alert prima del Restore e, nei 50 episodi valutati, non ha mai eseguito un intervento risolutivo tramite Restore o Isolate su un nodo di servizio o sul nodo critico mentre erano compromessi. Negli episodi in cui l'infezione supera le postazioni utente, questo schema di risposta non risulta sufficiente.

DQN mostra una tendenza simile a concentrare le azioni sulle postazioni utente, ma presenta una quota molto più elevata di azioni non valide e non ottiene vittorie nei 50 episodi considerati.

7.3 Livello 3 — Analisi di episodi interi

Un risultato in particolare è indicativo: cercando per PPO l'episodio difficile gestito meglio, il selettore non ha individuato vittorie tra gli episodi classificati come difficili e ha quindi restituito la sconfitta meno grave. Questo esito mostra che, secondo il criterio utilizzato per la selezione, le vittorie di PPO ricadono nei casi in cui l'infezione viene contenuta nelle fasi iniziali. Inoltre, gli episodi in cui il nodo critico viene compromesso ($\approx 12\%$) coincidono quasi completamente con le sconfitte totali ($\approx 12\%$).

7.4 Sintesi dell'analisi

Considerando insieme i tre livelli di analisi, PPO sceglie la stessa azione dell'euristica solo nel 2,1% delle mosse. La policy appresa segue quindi un comportamento diverso e più semplice, centrato sul ripristino delle postazioni utente che generano un alert. Nei 50 episodi valutati questo schema risulta efficace soprattutto quando l'infezione viene contenuta all'origine; non emerge invece una gestione efficace dei casi in cui l'attacco raggiunge nodi di servizio o il nodo critico. La robustezza rispetto a topologie o punti di ingresso differenti non è stata verificata sperimentalmente. DQN mostra una tendenza simile, con risultati nettamente peggiori nel protocollo considerato.

Il vantaggio di PPO sull'euristica è statisticamente significativo (Sezione 6); l'analisi comportamentale mostra tuttavia che tale risultato è ottenuto mediante una policy relativamente semplice, fortemente centrata sulle postazioni utente. La sola reward media non è quindi sufficiente a caratterizzare il comportamento appreso.

8 Discussione e limiti

L'analisi del comportamento di PPO consente di interpretarne il risultato e di individuare i principali limiti entro cui esso può essere considerato affidabile.

Il primo limite riguarda la dipendenza del comportamento appreso dalla struttura dello scenario. PPO sfrutta il fatto che il primo nodo infetto sia sempre una postazione utente e concentra la propria risposta sui relativi alert. Nei 50 episodi analizzati non sono stati osservati interventi risolutivi su nodi di servizio o sul nodo critico mentre erano compromessi. L'efficacia della policy con topologie diverse o con punti di ingresso differenti non è stata verificata sperimentalmente. Per valutarne la capacità di generalizzazione sarebbero quindi utili una topologia diversa e attacchi inizializzati anche su nodi di servizio o sul nodo critico.

Il secondo limite riguarda l'attaccante, volutamente semplice e non strategico: una policy che risulta efficace contro un'infezione che si propaga in modo casuale potrebbe rivelarsi inefficace contro un avversario che punta direttamente al nodo critico.

Il terzo limite riguarda i coefficienti della reward: i loro valori sono una scelta di modellazione che influisce sul comportamento appreso, e valori diversi avrebbero potuto portare a una strategia diversa. Il loro effetto non è stato esplorato in modo sistematico.

Infine, le prestazioni di DQN vanno interpretate nel contesto del protocollo adottato: l'algoritmo è stato valutato con iperparametri sostanzialmente di default e senza un tuning dedicato, per cui i risultati non consentono conclusioni sulla sua adeguatezza in generale.

La dimensione contenuta dell'ambiente e la topologia fissa hanno facilitato l'interpretazione del comportamento degli agenti e il collegamento tra decisioni osservate e caratteristiche dello scenario. In un ambiente più grande la stessa analisi sarebbe più complessa e i risultati più difficili da interpretare.

9 Conclusioni

Si riprendono qui le tre domande di partenza.

Un agente RL supera le strategie di riferimento? Nel confronto finale, PPO supera l'euristica con una differenza statisticamente significativa; per DQN non emerge invece un miglioramento statisticamente significativo rispetto alla strategia passiva.

Quale algoritmo si adatta meglio? Nel protocollo adottato, PPO ottiene risultati migliori e mostra un andamento dell'addestramento più stabile. Gli esperimenti svolti non consentono tuttavia di isolare una singola causa del divario rispetto a DQN.

Che cosa ha appreso effettivamente l'agente migliore? Il comportamento di PPO è dominato dal ripristino delle postazioni utente che generano un alert. Questa regola risulta efficace soprattutto quando l'infezione viene contenuta all'origine, mentre nei casi difficili osservati non emerge una gestione efficace dei nodi di servizio o del nodo critico compromessi. La robustezza della policy in condizioni differenti non è stata verificata sperimentalmente.

Il progetto combina quindi la valutazione quantitativa con un'analisi comportamentale dei due agenti, mostrando che la sola reward media non descrive completamente la policy appresa.

Restano aperte alcune possibili estensioni. La più diretta è valutare PPO in condizioni non coperte dagli esperimenti attuali: con una topologia diversa, un primo nodo infetto appartenente anche ad altre categorie o un attaccante che miri al nodo critico invece di diffondersi casualmente. Questi esperimenti permetterebbero di verificare se il comportamento appreso si generalizzi oltre lo scenario considerato. Si potrebbero inoltre studiare sistematicamente l'effetto dei coefficienti della reward e il comportamento su reti più grandi.

A Struttura del codice e riproducibilità

Il progetto è organizzato per responsabilità: l'ambiente, le strategie, gli algoritmi, gli strumenti di analisi e i test sono mantenuti in cartelle distinte. La Tabella 7 ne riporta la struttura essenziale. Gli strumenti di analisi sono separati dal nucleo dell'ambiente, così da ridurre il rischio di modifiche accidentali e lasciare ai test il compito di rilevare eventuali regressioni.

Tabella 7: Mappa dei file principali e loro responsabilità.

File	Responsabilità
<i>Ambiente</i>	
<code>config.py</code>	parametri globali dell'ambiente (rete, rumore, reward, episodio)
<code>topology.py</code>	definizione statica e dinamica dei nodi, costruzione della rete
<code>incident_response_env.py</code>	ambiente Gymnasium: reset, osservazione, azioni, dinamica, reward
<i>Strategie di riferimento</i>	
<code>policy_base.py</code>	interfaccia comune a tutte le strategie
<code>do_nothing.py</code> , <code>panic_isolation.py</code>	baseline passiva e iperprotettiva
<code>heuristic_defender.py</code> <code>oracle_defender.py</code>	strategia procedurale a priorità (riferimento principale) riferimento con conoscenza perfetta
<i>Agenti RL</i>	
<code>sb3_wrapper.py</code>	adattatore fra i modelli Stable-Baselines3 e il codice comune
<code>train_dqn.py</code> , <code>train_ppo.py</code>	addestramento dei due algoritmi
<i>Valutazione e analisi</i>	
<code>evaluate.py</code>	protocollo di valutazione comune a tutte le strategie
<code>bootstrap_ci.py</code>	intervalli di confidenza appaiati sulla differenza di reward
<code>analyze_policy.py</code>	analisi comportamentale (i tre livelli della Sezione 7)
<code>extract_action_dist.py</code> , <code>build_summary.py</code> , <code>plot_training_curves.py</code>	estrazione numeri, sintesi, curve
<i>Test</i>	
<code>test_env.py</code> , <code>test_baselines.py</code> , <code>conftest.py</code>	suite di verifica dell'ambiente e delle strategie

Riproduzione dei risultati. Per i principali risultati riportati, l'output del relativo comando viene salvato su file. In questo modo tali risultati possono essere ricostruiti a posteriori senza rieseguire gli addestramenti. I passaggi principali sono i seguenti (percorsi indicativi).

```
# 1. Addestramento (5 seed per algoritmo, 500k passi ciascuno)
python train_dqn.py --mode full --seeds 0,1,2,3,4 --normalize-reward
python train_ppo.py --mode full --seeds 0,1,2,3,4

# 2. Confronto finale su 50 episodi (seed 42): tabella + log per episodio
mkdir -p runs/final_comparison
python evaluate.py \
    --policy oracle,ppo,heuristic,dqn,do_nothing,random,panic_isolation \
    --model-path runs/dqn/full/seed1/best_model/best_model.zip \
    --ppo-model-path runs/ppo/full/seed3/best_model/best_model.zip \
    --seed 42 --n-episodes 50 --quiet --no-color \
    --save --save-dir runs/final_comparison \
    > runs/final_comparison/comparison_final.txt
```

```

# 3. Significativita' statistica (bootstrap appaiato): un file per coppia
mkdir -p runs/final_comparison/significance
python tools/bootstrap_ci.py \
    --a runs/final_comparison/evaluations/eval_ppo_best_model.zip__seed42_summaries.jsonl \
    --b runs/final_comparison/evaluations/eval_heuristic_seed42_summaries.jsonl \
    --label-a PPO --label-b heuristic \
    > runs/final_comparison/significance/ppo_vs_heuristic.txt
# (ripetere per: ppo_vs_dqn, ppo_vs_do_nothing, dqn_vs_heuristic, dqn_vs_do_nothing)

# 4. Analisi della policy appresa (un'unica esecuzione, entrambi gli agenti)
mkdir -p runs/policy_analysis
python tools/analyze_policy.py \
    --ppo-model-path runs/ppo/full/seed3/best_model/best_model.zip \
    --dqn-model-path runs/dqn/full/seed1/best_model/best_model.zip \
    --seed 42 --n-episodes 50 --no-color \
    --out-dir runs/policy_analysis \
    > runs/policy_analysis/policy_analysis.txt
# genera anche key_numbers.json e representative_episodes_{ppo,dqn}.txt
# dentro runs/policy_analysis/

# 5. Curve di training (in runs/final_comparison/)
python tools/plot_training_curves.py

```

B Iperparametri completi

DQN (default Stable-Baselines3 2.8.0, tranne *learning starts*): learning rate 1×10^{-4} , replay buffer 1 000 000, learning starts 5 000, batch size 32, $\tau = 1,0$, $\gamma = 0,99$, train freq 4, gradient steps 1, target update ogni 10 000 passi, esplorazione da $\varepsilon = 1,0$ a 0,05 (frazione 0,1), max grad norm 10. Reward normalizzata ($\div 17$).

PPO (default Stable-Baselines3 2.8.0): learning rate 3×10^{-4} , n_steps 2048, batch size 64, n_epochs 10, $\gamma = 0,99$, GAE $\lambda = 0,95$, clip range 0,2, coeff. entropia 0,0, coeff. value function 0,5, max grad norm 0,5. Reward non normalizzata (i vantaggi sono normalizzati internamente).

Comune: 500 000 passi per seed, 5 seed (0–4), valutazione periodica ogni 25 000 passi su 50 episodi, politica MLP.

C File di output salvati

Per le principali fasi di valutazione e analisi, gli output sono stati conservati su file in modo da poter ricostruire i risultati senza rieseguire gli addestramenti. I principali sono: la tabella di confronto finale (`comparison_final.txt`) e la sua versione sintetica (`synthesis.txt`); gli intervalli di confidenza appaiati; i numeri chiave dell'analisi in forma leggibile (`policy_analysis.txt`) e strutturata (`key_numbers.json`); gli episodi rappresentativi commentati; le curve di training (sia le immagini sia il riepilogo numerico per seed); e, per ogni addestramento, un file di metadati con seed, iperparametri, versione delle librerie e tempi.